

```

        square[new_entry_i][new_entry_j] = rule3(square[entry_i]
[entry_j],n,True)

        if(where_to_go == 'down_right' and (entry_i + entry_j) == (n-2)):
            new_entry_i = entry_i + 1
            new_entry_j = entry_j + 1
            square[new_entry_i][new_entry_j] = rule3(square[entry_i]
[entry_j],n,False)

        ❶ entry_i = new_entry_i
        entry_j = new_entry_j

    return(square)

```

Listing 2-2: *A function that enables an implementation of Kurushima’s algorithm*

This function will take four arguments: first, a starting square that has some `nan` entries; second and third, the indices of the entry that we want to start with; and fourth, how much we want to fill up the square (measured by the number of `nan` entries we are willing to tolerate). The function consists of a `while` loop that writes a number to an entry in the square at every iteration by following one of our three rules. It continues until it has as many `nan` entries as we have specified in the function’s fourth argument. After it writes to a particular entry, it “travels” to that entry by changing its indices ❶, and then it repeats again.

Now that we have this function, all that remains is to call it in the right way.

Using the Right Arguments

Let’s start with the central entry and fill up the magic square from there. For our `howfull` argument, we’ll specify $(n^2)/2-4$. The reason for using this value for `howfull` will become clear after we see our results:

```

entry_i = math.floor(n/2)
entry_j = math.floor(n/2)

square = fillsquare(square,entry_i,entry_j,(n**2)/2 - 4)

```
